

Article

The Arithmetic Jump: A Branch-Free Index Inversion for 3D Arrays

Paul A. Gagniuc 

Faculty of Engineering in Foreign Languages, National University of Science and Technology Politehnica
Bucharest, RO-060042 Bucharest, Romania; paul.gagniuc@upb.ro

Abstract

This work presents a compact arithmetic formulation for inverting row-major linear indices into three-dimensional coordinates. The formulation defines a bijective and reversible mapping based solely on integer division and modulo operations and avoids iteration and control-flow constructs. A traversal-based reconstruction strategy and the arithmetic formulation are evaluated on Graphics Processing Unit (GPU) hardware across multiple volumetric configurations. The experimental results show that arithmetic index decomposition yields uniform execution behavior, low run-to-run timing variability, and constant per-thread execution cost under massively parallel execution. The observed differences follow from GPU architectural characteristics, particularly sensitivity to control-flow divergence. The formulation provides a portable reference model for multidimensional index inversion suitable for parallel kernels and hardware-oriented implementations.

Keywords: multidimensional indexing; CUDA optimization; loop flattening; GPU kernels; memory access

1. Introduction

The traversal and indexing of multidimensional arrays constitute a foundational aspect of both theoretical computer science and practical programming [1,2]. Since early programming environments such as FORTRAN and assembly languages, multidimensional data structures have been represented as contiguous linear memory regions, which necessitated systematic mappings between linear storage and multidimensional coordinate systems [3,4]. These mappings form the basis of array access semantics in virtually all programming languages and computational models.

Historically, multidimensional traversal has often been expressed through nested iteration constructs, with each loop corresponding to one array dimension. While this representation aligns naturally with mathematical abstractions such as matrices and tensors, it intertwines data layout with control flow and introduces a structural overhead that becomes increasingly relevant in low-level, performance-critical, or hardware-oriented contexts [5–7]. In early compiled languages, developers and compilers frequently applied manual or heuristic optimizations to mitigate loop overhead and improve memory locality, including loop unrolling, reordering, and cache-aware transformations [8–14]. Such optimizations, however, are typically environment-dependent and do not alter the underlying indexing semantics.

Modern programming languages and numerical frameworks further abstract array access through high-level constructs and vectorized operations [1,15,16]. Arrays are commonly represented as objects enriched with metadata such as dimensionality, shape vectors, and stride information [17–20]. While these abstractions improve expressiveness and



Academic Editor: Sándor Szénási

Received: 19 April 2026

Revised: 6 May 2026

Accepted: 7 May 2026

Published: 11 May 2026

Copyright: © 2026 by the author.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

portability, the fundamental act of data access remains governed by arithmetic index transformations that translate between linear memory addresses and multidimensional coordinates [21]. As data dimensionality and scale increase, such as in deep learning workloads, volumetric imaging, or voxel-based simulations, the efficiency, predictability, and generality of these transformations become increasingly important [22].

The purpose of this work is to present a formal and explicit arithmetic formulation of linear-to-multidimensional index inversion for arrays stored in row-major order. This study focuses on the canonical bijective mapping between a linear index and its corresponding coordinate tuple under row-major layout assumptions. The mapping admits a minimal, branch-free arithmetic formulation that relies exclusively on integer division and modulo operations [23,24]. The formulation avoids control-flow constructs, nested iteration, and auxiliary lookup structures, making it particularly suitable for environments where deterministic execution and structural simplicity are required. The central observation is that, for a row-major flattened array, the inverse mapping from a linear index to a coordinate vector admits a closed-form arithmetic representation. This representation is independent of programming language, runtime environment, or execution model. As a result, the same formulation can be instantiated consistently across software implementations (e.g., C++, Python, JavaScript), parallel kernels (e.g., Compute Unified Device Architecture (CUDA)), compiler backends, and even hardware description or logic simulation environments such as Verilog or Logisim Evolution. In such contexts, the elimination of branches and loop constructs can simplify control logic, facilitate parallel execution, and enable predictable timing behavior. Thus, this work provides a unified and explicit arithmetic formulation of linear index inversion, a mechanism that underlies array access in compilers, libraries, and runtime systems. Although the mapping corresponds to the standard row-major index inversion, the formulation presented here expresses it in an explicit modular normal form that isolates control-free arithmetic suitable for parallel and hardware-oriented execution. Thus, the formulation serves as a portable reference model for software implementations, parallel execution contexts, and hardware-oriented designs. It emphasizes determinism, structural minimalism, and control-free execution in index transformation.

2. Materials and Methods

A row-major three-dimensional array admits a deterministic arithmetic correspondence between each linear index and a unique coordinate triplet. This correspondence relies exclusively on integer division and modulo operations and yields an exact inverse of the row-major flattening rule. The mapping is bijective, reversible, and free of control-flow constructs, which ensures consistent behavior under parallel execution. Parallel execution is important in this context because large multidimensional arrays contain many elements that can be processed independently. Thus, a direct index reconstruction formula allows each thread to recover its own coordinate triplet without communication with other threads, shared counters, or sequential traversal.

2.1. A Deterministic Arithmetic Mapping

Let $A \in \mathbb{R}^{m \times n \times s}$ denote a three-dimensional array with m rows, n columns, and s depth layers. Let $q = m \times n \times s$ represent the total number of elements. For any linear index $v \in [0, q - 1]$, there exists a unique coordinate triplet $(i, j, d) \in \mathbb{N}^3$ such that the element at position v corresponds to $A[i][j][d]$. The mapping from v to (i, j, d) follows the arithmetic system as seen in Equations (1)–(4):

$$k = v \bmod (m \times n) \quad (1)$$

$$j = v \bmod n \quad (2)$$

$$i = \frac{k-j}{n} \quad (3)$$

$$d = \frac{v-k}{m \times n} \quad (4)$$

This transformation is total and deterministic for all integers $v \in [0, mns - 1]$. It provides an exact inversion of the row-major flattening process, where the column index j varies fastest, followed by the row index i , and finally the depth index d .

2.2. Proof, Injectivity, and Reversibility

A three-dimensional array $A \in \mathbb{R}^{m \times n \times s}$ admits a row-major linear representation of length $q = m \times n \times s$, where each element $A[i][j][d]$ maps to a linear index v according to:

$$v = j + i \times n + d \times (m \times n) \quad (5)$$

This expression defines an injective mapping from the coordinate space $(i, j, d) \in [0, m-1] \times [0, n-1] \times [0, s-1]$ to the linear index domain $[0, q-1]$. In order to recover the original coordinates from v , the transformation isolates each component through modular reduction and integer division. The value $k = v \bmod (m \times n)$ yields the offset within the corresponding two-dimensional slice. The column index follows directly as $j = k \bmod n$, since j represents the fastest-varying dimension. The row index results from $i = (k - j)/n$, and the depth index follows as $d = (v - k)/(m \times n)$. All operations involve exact integer arithmetic. Given the bounds on v , and the divisibility properties of n and $m \times n$, the expressions yield unique integers $i \in [0, m-1]$, $j \in [0, n-1]$, and $d \in [0, s-1]$. Thus, each linear index corresponds to exactly one coordinate triplet, and each coordinate triplet maps to exactly one linear index. The transformation is bijective and reversible, which concludes the proof (Figure 1).

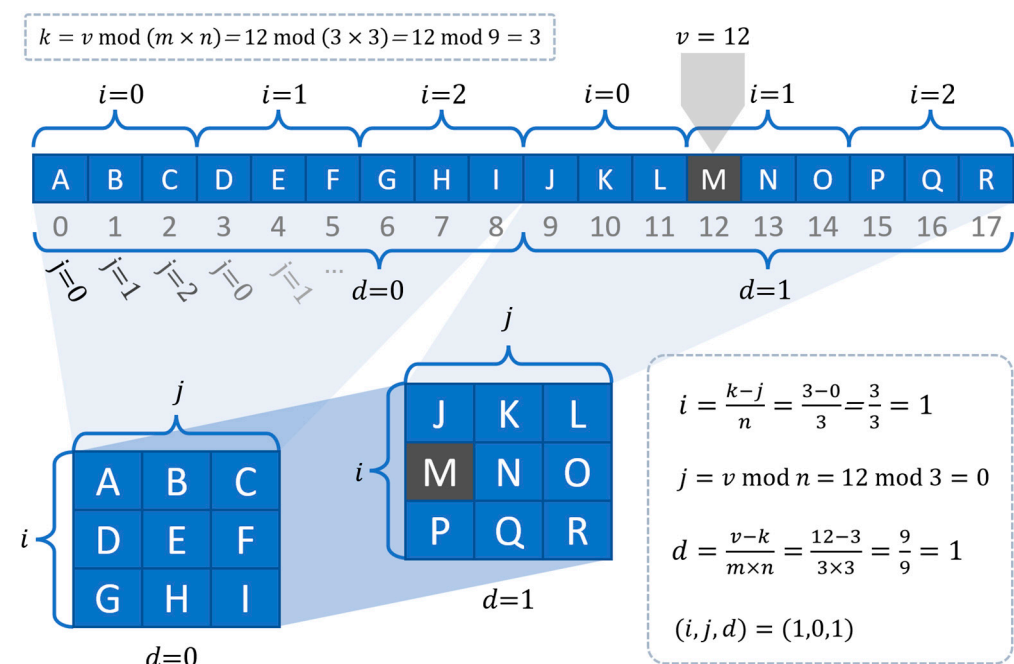


Figure 1. Arithmetic decomposition of a linear index. The example uses a $3 \times 3 \times 2$ row-major array with zero-based indexing and selects $v = 12$. The three-dimensional object is represented as two 2D slices, each slice with 3×3 elements. The first slice corresponds to $d = 0$, and the second slice corresponds to $d = 1$. The dark cell marks the selected linear position, namely index 12, labeled “M”. The value $k = 3$ represents the offset of this index inside the current slice. Since each slice contains nine elements, index 12 belongs to the second slice, so the depth coordinate is $d = 1$. Inside that slice, the selected cell is located on row $i = 1$ and column $j = 0$. Therefore, the selected linear index corresponds to the coordinate triplet $(i, j, d) = (1, 0, 1)$. The example illustrates direct coordinate recovery without nested traversal.

2.3. Forward Index Mapping

For completeness, the forward transformation from coordinates $(i, j, d) \in \mathbb{N}^3$ to linear index $v \in [0, mns - 1]$ follows directly from the row-major layout rule:

$$v = j + i \times n + d \times (m \times n) \quad (6)$$

This arithmetic expression computes the linear position of any coordinate triplet through a single additive sequence. Together with the inverse mapping defined above, it establishes a complete bidirectional correspondence between linear and multidimensional index spaces (Figure 1).

2.4. Mathematical Validity

The arithmetic mapping relies exclusively on modular arithmetic and integer division, and establishes a one-to-one correspondence between the linear index space and the multidimensional coordinate space. Injectivity follows from the non-overlapping weight structure of the forward mapping: j contributes to the unit term, i scales by n , and d scales by $m \times n$. Given the bounds on each index, no two distinct coordinate triplets can map to the same linear index. Reversibility follows from the closed-form expressions that recover each coordinate directly from v . Each component derives independently through exact arithmetic operations, without iteration, recursion, or conditional logic. This arithmetic stability ensures consistent results across all valid inputs. Therefore, the index model is minimal, deterministic, and branch-free. These main properties support use in software implementations, parallel execution contexts, and hardware-level address translation, particularly where predictable execution behavior and regular memory access patterns are required.

2.5. Experimental Design

In order to evaluate the index-mapping strategies in GPU-based systems, a dedicated benchmarking framework was implemented in CUDA C++. The evaluation compares two approaches for the decomposition of a linear index into three-dimensional coordinates within volumetric data: an iterative traversal method and a direct arithmetic method. Two CUDA kernels were implemented. The first kernel, referred to as the traversal method, iterates over the full 3D volume through three nested loops over depth, height, and width. A counter variable increments until it equals the linear index of the thread, at which point the corresponding coordinates are recorded. This method exhibits linear-time complexity and introduces control-flow branching. The traversal-based kernel is intentionally implemented as a procedural scan of the index space in order to isolate the effects of control-flow divergence under GPU execution, rather than to represent an optimized index reconstruction strategy (Supplementary Material). The second kernel implements the arithmetic decomposition through integer division and modulo operations (Algorithm 1):

```
int k = idx % (width * height);
int j = k % width;
int i = (k - j) / width;
int d = (idx - k) / (width * height);
```

Algorithm 1. One loop traversal of a 3D array by arithmetic decomposition

```

Input: dimensions  $m, n, s$ 
Output: coordinates  $i, j, d$  for each linear index  $v$ 
For each parallel thread with linear index  $v$ , where  $0 \leq v < m \times n \times s$ , do
     $k \leftarrow v \bmod (m \times n)$ 
     $j \leftarrow v \bmod n$ 
     $i \leftarrow (k - j)/n$ 
     $d \leftarrow (v - k)/(m \times n)$ 
    process  $A[i][j][d]$ 
End For

```

This formulation computes the coordinates in constant time and avoids control-flow divergence. Each thread performs an identical arithmetic sequence, which supports parallel execution across the full index domain. Ten volumetric configurations were evaluated, with both cubic and non cubic shapes. The evaluated configurations were: $16 \times 16 \times 16$, $32 \times 32 \times 32$, $64 \times 64 \times 64$, $128 \times 128 \times 128$, $256 \times 256 \times 64$, $512 \times 128 \times 64$, $512 \times 256 \times 64$, $1024 \times 256 \times 64$, $1024 \times 512 \times 32$, and $1024 \times 1024 \times 16$. Each configuration underwent seven independent runs. Memory allocation used *cudaMallocManaged* for all output arrays. Kernel launches used a block size of 256 threads and sufficient blocks to cover the entire index range. Execution synchronization followed each kernel launch via:

cudaDeviceSynchronize();

Timing measurements relied on *std::chrono::high_resolution_clock* on the host side. Each kernel populated coordinate arrays for validation. A host-side verification step compared outputs from both methods across all indices. Matching results confirmed correctness for each configuration. Performance metrics included execution time in milliseconds, average runtime across runs, standard deviation, speedup ratios, and relative improvement percentages.

2.6. Hardware and Software Configuration

All experiments ran on a Dell Precision Tower 3620 workstation with an Intel Core i7-7700 CPU at 3.60 GHz, 64 GB of RAM, and Microsoft Windows 10 Pro 64-bit (Build 19045). GPU execution used an NVIDIA Quadro K2200 with 640 CUDA cores, 4 GB GDDR5 memory, a 128-bit memory interface, and 80.16 GB/s memory bandwidth. CUDA Toolkit 12.8 Update 1, NVIDIA driver version 572.83, and Visual Studio 2022 Build Tools with MSVC v143 were used for compilation and execution. Kernel launches followed the configuration $\lll(\text{TOTAL} + 255)/256, 256\ggg$ across all tests. Benchmarking covered total element counts from 4,096 to over 16 million. Each configuration underwent seven independent trials to ensure statistical consistency. All runs are executed directly on physical GPU hardware without virtualization or throttling.

3. Results

Performance evaluation covered two index-mapping strategies on ten three-dimensional grid configurations. Reported execution times represent mean values across repeated runs for each configuration. The evaluated configurations are summarized in Table 1 for direct comparison. The comparison examined execution time, speedup ratios, percentage improvement, and temporal variability (Appendix A). Across all tested configurations, the arithmetic index method shows a clear and consistent execution-time advantage. Average kernel runtimes, summarized in Figure 2c, remain lower for the arithmetic formulation in every case. Performance differences are quantified through two metrics: the speedup ratio,

defined as the mean execution time of the traversal-based method divided by that of the arithmetic method, and the percentage improvement, which reflects the relative reduction in execution time. Speedup ratios range between approximately $10\times$ and $17.5\times$ for most configurations (Table 1). For the $1024 \times 512 \times 32$ grid, the arithmetic method achieved a $17.46\times$ speedup, while the $1024 \times 256 \times 64$ configuration yielded a $14.29\times$ speedup.

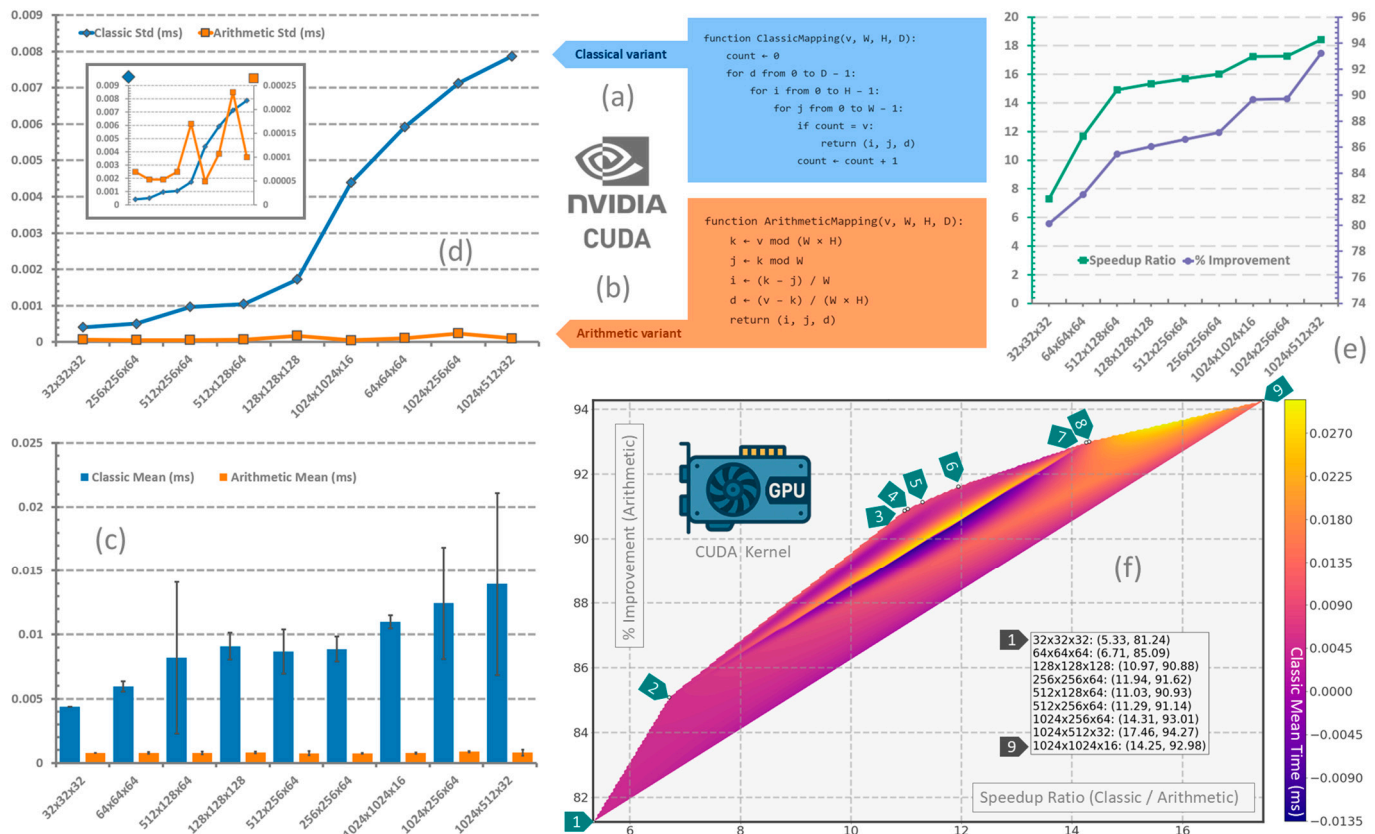


Figure 2. Performance comparison between traversal-based and arithmetic index mapping on GPU hardware. Panel (a) shows pseudocode for the traversal-based (“Classic”) variant, which derives three-dimensional coordinates through triple nested loops and counter matching. Panel (b) shows pseudocode for the arithmetic variant, which computes coordinates directly through integer division and modulo operations. Panel (c) reports mean kernel execution time (ms) across tested grid dimensions, with error bars indicating standard deviation. Panel (d) shows the corresponding standard deviation of execution time, including an inset that magnifies low-variance regions. It reports the standard deviation values separately from the mean execution times. The configurations are ordered by increasing Classic standard deviation, in order to show the growth of variability in the traversal-based method. The inset uses a secondary scale to make the smaller Arithmetic standard deviation values visible. Panel (e) presents the speedup ratio (Classic divided by Arithmetic) and the percentage reduction in execution time for the arithmetic method. Panel (f) summarizes performance across configurations, plotting speedup ratio versus percentage reduction, with color indicating execution time of the traversal-based variant. Numbered markers indicate the evaluated grid configurations.

These results confirm the computational efficiency of direct arithmetic index decomposition for practical three-dimensional workloads. Figure 2e summarizes the speedup and improvement trends across all dimensions. The percentage improvement metric reinforces the same observation. Values exceed 90% in all non-trivial configurations, which indicates that over 90% of execution time is eliminated through direct arithmetic decomposition. The $128 \times 128 \times 128$ grid showed a 90.88% reduction, while the $1024 \times 512 \times 32$ grid reached a peak reduction of 94.27% (Table 1).

Table 1. Performance comparison between traversal-based and arithmetic 3D index decomposition on GPU hardware. Each row corresponds to a tested volumetric configuration. Execution times are reported in milliseconds. The table lists mean execution time, standard deviation, speedup ratio (traversal-based divided by arithmetic), and percentage reduction in execution time for the arithmetic method. Bold values mark the minimum (82.0) and maximum (94.3) values.

Dimensions	Classic Mean $\pm$ Std (ms)	Arithmetic Mean $\pm$ Std (ms)	Speedup Ratio *	% Improvement Arithmetic
$1024 \times 1024 \times 16$	0.010986 ± 0.004387	$0.000771 \pm 4.9 \times 10^{-5}$	14.24	92.97789
$1024 \times 256 \times 64$	0.012457 ± 0.007125	0.000871 ± 0.000236	14.29	93.00459
$1024 \times 512 \times 32$	0.013971 ± 0.007870	0.000800 ± 0.000100	17.46	94.27403
$128 \times 128 \times 128$	0.009086 ± 0.001727	0.000829 ± 0.000170	10.96	90.88050
$16 \times 16 \times 16$	0.934543 ± 0.049906	$0.001314 \pm 9.0 \times 10^{-5}$	711.0	[†] 99.85937
$256 \times 256 \times 64$	0.008871 ± 0.000506	$0.000743 \pm 5.3 \times 10^{-5}$	11.94	91.62641
$32 \times 32 \times 32$	0.004371 ± 0.000407	$0.000786 \pm 6.9 \times 10^{-5}$	5.563	82.02614
$512 \times 128 \times 64$	0.008186 ± 0.001046	$0.000786 \pm 6.9 \times 10^{-5}$	10.41	90.40140
$512 \times 256 \times 64$	0.008671 ± 0.000971	$0.000757 \pm 5.3 \times 10^{-5}$	11.45	91.26853
$64 \times 64 \times 64$	0.005971 ± 0.005919	0.000786 ± 0.000107	7.600	86.84211

* (Classic/Arithmetic). [†] The $16 \times 16 \times 16$ configuration is reported as a small dimension outlier.

3.1. Small-Dimension Outlier

The $16 \times 16 \times 16$ configuration is reported for completeness only and is explicitly excluded from quantitative comparison and trend analysis, as the traversal-based kernel degenerates into a full linear scan per thread, which produces trivial speedups dominated by algorithmic imbalance rather than architectural effects. Although the workload contains only 4096 elements, the traversal-based kernel reports an execution time of approximately 0.935 ms, while the arithmetic kernel remains below 0.015 ms. This discrepancy yields a speedup above 700 $\times$ and a percentage improvement of 99.86% (Table 1). The anomaly reflects algorithmic imbalance in the traversal-based kernel, with fixed GPU overhead effects that become visible only at this scale, rather than a measurement artifact. Accordingly, this configuration is excluded from trend interpretation but retained for completeness.

3.2. Temporal Stability

The arithmetic index method exhibits strong temporal stability across all configurations. Standard deviation values remain consistently low, typically below 0.0002 ms (Table 1), as illustrated in Figure 2d. In contrast, the traversal-based method shows greater variance, particularly for lower-dimensional grids. This variability follows from sensitivity to control flow, memory access order, and kernel launch overhead. Execution time for the arithmetic method remains nearly constant as data volume increases from mid-scale grids such as $128 \times 128 \times 128$ to large configurations such as $1024 \times 512 \times 32$. This behavior reflects constant-time index decomposition per thread and supports scalability under high-throughput conditions. The performance trajectory across all tested geometries appears in Figure 2f.

3.3. Timing Behavior at Small Scales

The performance disparity observed at small array sizes follows directly from the structural properties of the traversal-based method. Each GPU thread executes a sequential search through the multidimensional domain until it matches the target index. This procedure incurs linear time complexity with respect to the total element count (Table 2). The

arithmetic method, by contrast, derives all coordinate components through fixed arithmetic expressions. Each thread executes an identical instruction sequence with constant time complexity. At small scales, kernel launch overhead and synchronization costs dominate absolute execution time. These fixed costs penalize traversal-based logic more severely due to its inherent inefficiency. This behavior highlights a central property of arithmetic index decomposition: the method avoids control-flow penalties at small scales while maintaining scalability as array dimensions increase.

Table 2. Qualitative comparison of 3D index decomposition methods. The comparison covers control-flow structure, branching behavior, memory access characteristics, and execution properties.

Criterion	Traversal-Based	Arithmetic
Index derivation	Nested traversal	Arithmetic formula
Control flow	Data-dependent	Uniform
Branching	Required	Absent
Execution cost	Index-dependent	Index-independent
Parallel behavior	Divergence possible	Warp-uniform

4. Discussion

The arithmetic index method presented here defines a deterministic and purely arithmetic alternative to loop-based traversal of multidimensional arrays. The discussion highlights four main aspects of the proposed method: GPU relevance, mathematical value, execution behavior, and architectural implications. The formulation relies exclusively on integer division and modulo operations and admits direct implementation across software and hardware platforms. Its structure is minimal, composable, and independent of control flow, which suits environments where conditional logic or iterative constructs incur high cost.

On general-purpose CPUs and in high-level languages such as JavaScript and C++, compiler-optimized loop-based traversal can outperform direct arithmetic decomposition, primarily due to aggressive loop optimization and the higher latency of integer division and modulo operations. This outcome reflects decades of compiler and runtime optimization rather than a limitation of the arithmetic formulation itself. More specifically, modern compilers aggressively optimize nested loops through unrolling, instruction reordering, branch prediction, and speculative execution. Speculative execution and scheduling studies further show that loop-based performance depends on execution order, chunk size, and runtime scheduling decisions [25,26]. These optimizations reduce loop control overhead, improve cache locality, and allow better use of CPU pipelines. GPU studies also show that performance is shaped by how kernels assign work to threads and by how memory is accessed during execution [27–31]. As a result, loop-based traversal may remain efficient on CPUs, even when a direct arithmetic formulation is available. Arithmetic operations that involve division and modulo receive more conservative treatment and often resist vectorization. Under sequential execution on scalar processors, these architectural factors favor loop constructs. For this reason, the traversal-based method serves as a meaningful baseline rather than a strawman. It represents the canonical pedagogical and historical approach to multidimensional traversal and reflects the form that compilers and runtimes optimize most aggressively. The use of three nested loops also mirrors how multidimensional iteration appears in teaching material, reference implementations, and legacy code. Thus, the comparison shows how arithmetic index reconstruction interacts with massively parallel execution models. The arithmetic method does not target maximum speed on general-purpose CPUs. Its primary advantage lies in uniform, branch-free execution rather than scalar instruction throughput. This property aligns with parallel processors and hard-

ware logic, where predictable execution and control-flow simplicity determine performance. Related hardware and heterogeneous computing studies further emphasize the importance of predictable execution paths across FPGA, CPU, and GPU systems [32–38]. Moreover, the two index decomposition strategies differ in control-flow structure and execution behavior. A structural comparison of traversal-based and arithmetic index decomposition methods is provided in Table 2.

4.1. GPU and Hardware Relevance

The principal advantage of arithmetic index decomposition emerges on GPUs and programmable logic devices. GPUs favor uniform, branch-free instruction streams and penalize divergent control paths. Each GPU thread executes a simple instruction sequence, and warp-level efficiency depends on identical control flow across threads. Loop-based traversal introduces divergence because threads reach their target index after different iteration counts. This divergence forces serialization within warps and reduces throughput. The arithmetic method avoids this penalty. Warp-level execution studies also show that branch behavior and warp scheduling have a direct effect on GPU throughput [39,40]. Each thread computes its coordinates through the same fixed arithmetic expressions and completes in constant time. The absence of control flow preserves lockstep execution and maximizes parallel efficiency. These properties explain the strong performance and stability observed in the experimental results. Field-programmable gate arrays and custom logic circuits impose even stricter constraints. Hardware synthesis favors combinational arithmetic and pipeline-friendly logic over stateful iteration and branching. The arithmetic formulation maps directly onto such architectures, where coordinate reconstruction can occur through fixed datapaths without counters, comparators, or control units.

4.2. Mathematical and Systems-Level Value

Beyond execution speed, the method provides value as a formal model of index inversion. It expresses multidimensional structure through arithmetic alone and avoids mutable state, loop counters, and control dependencies. This property proves useful in symbolic computation, discrete geometry, tensor algebra, cellular automata, and related domains that require bijective mappings between linear and structured index spaces. Similar directions exist in predictive modeling, where explicit mathematical formulations replace adaptive learning strategies. Deterministic approaches based on direct transformations have been proposed as alternatives to classical neural networks (e.g., the spectral forecast method), with emphasis on interpretability and structural transparency [41,42]. Related challenges appear in spatial data systems, where large volumes of indexed positions require efficient access and comparison across datasets [43,44]. Compiler and language runtime design also benefit from such a formulation [45]. High-level array operations rely internally on index translation for slicing, reshaping, and broadcasting. An explicit arithmetic model reduces internal complexity and improves portability, even if it does not directly accelerate execution on scalar processors. The method also holds pedagogical value. It demonstrates how structure emerges from arithmetic relationships rather than iteration. This perspective bridges abstract algebraic reasoning and concrete system implementation and clarifies the distinction between traversal and index reconstruction.

4.3. Scope and Limitations

The arithmetic method does not replace loop-based traversal in all contexts. It does not aim to outperform compiler-optimized loops on CPUs under sequential execution. Its role lies elsewhere: environments that require deterministic timing, uniform execution paths, or formal index functions rather than procedural traversal.

4.4. Geometry Effects in GPU Execution

The benchmark suite deliberately includes grid pairs such as $1024 \times 256 \times 64$ and $1024 \times 512 \times 32$, which share the same total element count but differ in spatial geometry. This choice reflects GPU execution realities. Performance depends not only on data volume but also on memory layout, thread block mapping, and access regularity. Distinct geometries influence memory coalescence, cache utilization, and shared memory behavior. Inclusion of both configurations exposes these effects and avoids conflation of volume size with execution efficiency.

4.5. Comparative Characteristics

The traversal-based method exhibits linear-time behavior per thread, since each thread advances sequentially through the volume until it reaches its index. Execution cost increases with index magnitude and introduces divergence across threads. The arithmetic method computes the coordinates in constant time through fixed arithmetic expressions (Table 2). All threads follow the same instruction path, which preserves warp coherence and reduces scheduling overhead. Empirical measurements reflect these structural differences. The arithmetic method shows low timing variability across repeated runs, while the traversal-based method exhibits higher fluctuation and sensitivity to grid geometry.

4.6. Architectural Implications

The performance gap arises from fundamental architectural differences between CPUs and GPUs. CPUs contain a small number of complex cores with deep cache hierarchies and advanced branch handling. GPUs consist of many simple cores optimized for throughput under uniform control flow. Arithmetic index decomposition aligns naturally with the GPU execution model, while traversal-based logic aligns with CPU optimization strategies. The significance of the arithmetic method therefore lies not in loop elimination per se, but in control-flow elimination. Thus, the method removes control-flow divergence and enforces uniform execution across threads. These properties align with GPU architecture characteristics and provide consistent, scalable behavior for high-dimensional data structures such as voxel grids and tensor volumes.

5. Conclusions

This work presents an explicit arithmetic formulation for the inversion of row-major linear indices into three-dimensional coordinates. The formulation defines a bijective and reversible correspondence that relies solely on integer division and modulo operations and avoids control-flow constructs. The experimental results demonstrate stable execution behavior and consistent performance under massively parallel execution models. The observed differences between arithmetic and traversal-based strategies follow directly from architectural characteristics of GPU hardware, particularly sensitivity to control-flow divergence and uniformity of execution. The method does not aim to replace loop-based traversal in general-purpose CPU contexts. Its relevance lies in environments where deterministic behavior, structural simplicity, and uniform execution paths dominate design constraints, such as GPU kernels, hardware logic, and compiler-level index reconstruction. This work formalizes index inversion as a compact arithmetic transformation and provides a clear, portable reference model for multidimensional indexing. The formulation supports reproducible implementation across software and hardware platforms. It defines the index inversion mechanism in a compact and explicit arithmetic form. The present study also opens several extensions. The same arithmetic view can apply to arrays with more than three dimensions by use of cumulative dimension products. Other memory layouts, such as column major or blocked layouts, can also have related formulas, which would permit

direct comparison between layout policies. A further direction concerns tests on newer GPU architectures, where the latency of integer division and modulo operations may differ from the hardware used in this study. The formulation may also pass into FPGA or custom logic circuits, where fixed datapaths can replace sequential traversal. These extensions would clarify the role of arithmetic index inversion across software, parallel kernels, and hardware-level address generation.

Supplementary Materials: The following supporting information can also be downloaded at: <https://www.mdpi.com/article/10.3390/a19050375/s1>. <https://github.com/Gagniuc/The-Arithmetic-Jump> (accessed on 1 May 2026).

Funding: This research received no external funding.

Data Availability Statement: The source code and experimental outputs that support the findings of this study are available in the Supplementary Material and are also publicly accessible at: <https://github.com/Gagniuc/The-Arithmetic-Jump> (accessed on 1 May 2026).

Conflicts of Interest: The author declares no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

CPU	Central Processing Unit
GPU	Graphics Processing Unit
CUDA	Compute Unified Device Architecture
FPGA	Field-Programmable Gate Array
$O(1)$	Constant Time Complexity
$O(n)$	Linear Time Complexity
ms	Milliseconds

Appendix A

The Supplementary Material associated with this work contains the complete source code and execution outputs used in the experimental evaluation of the traversal-based and arithmetic index decomposition methods. The material is organized into three directories, each corresponding to a distinct stage of development and validation. These files are not only supplementary code, but also part of the validation record, since they document correctness checks, repeated runs, and extended tests across larger configurations.

Appendix A.1. Baseline Implementation and Initial Tests

The first directory contains the initial CUDA implementation of both index decomposition strategies. This version provides a minimal and transparent realization of the traversal-based kernel and the arithmetic kernel, along with a simple test harness. The executable produced from this source reports kernel execution times and verifies correctness by comparing the coordinate outputs of both methods for each tested configuration. These tests establish functional equivalence and serve as a baseline for subsequent refinements.

Appendix A.2. Fixed Trial Implementation with Repeated Measurements

The second directory includes a revised implementation that addresses measurement stability and repeatability. This version introduces multiple independent trials for each volumetric configuration and reports execution times for both kernels across repeated runs. The outputs demonstrate consistent correctness (“Result match: YES”) and provide empirical evidence of run-to-run timing behavior. This directory supports the analysis of execution-time variability reported in the Section 3.

Appendix A.3. Extended Experiments on Larger Configurations

The third directory contains an extended version of the benchmarking code designed to evaluate larger three-dimensional grid configurations and additional aspect ratios. This implementation follows the same arithmetic formulation and traversal logic but expands the tested parameter space. The recorded outputs confirm correctness across all tested dimensions and provide additional timing data for large-scale configurations. These experiments support the scalability discussion and illustrate behavior under increased workload size.

References

- Gagniuć, P.A. Traversal of Multidimensional Arrays. In *Coding Examples from Simple to Complex: Applications in Python*; Springer: Cham, Switzerland, 2024; pp. 59–70.
- Ma, D.; Hu, X.; Zhang, H.; Sun, Q.; Xie, X. A hierarchical event detection method based on spectral theory of multidimensional matrix for power system. *IEEE Trans. Syst. Man Cybern. Syst.* **2019**, *51*, 2173–2186. [\[CrossRef\]](#)
- Numrich, R.W.; Reid, J. Co-Array Fortran for parallel programming. *ACM Sigplan Fortran Forum* **1998**, *17*, 1–31. [\[CrossRef\]](#)
- Reale, F.M.; Barbera, S.; Sciortino, S. Fast parallel implementation of multidimensional data-domain FORTRAN codes on distributed-memory processor arrays. *Comput. Phys. Commun.* **1992**, *72*, 129–143. [\[CrossRef\]](#)
- Kanakagiri, R.; Solomonik, E. Minimum cost loop nests for contraction of a sparse tensor with a tensor network. In Proceedings of the 36th ACM Symposium on Parallelism in Algorithms and Architectures, Nantes, France, 17–21 June 2024.
- Kodosky, J.; MacCracken, J.; Rymar, G. Visual programming using structured data flow. In *Proceedings 1991 IEEE Workshop on Visual Languages*; IEEE Computer Society: Kobe, Japan, 1991.
- Beck, M.; Johnson, R.; Pingali, K. From control flow to dataflow. *J. Parallel Distrib. Comput.* **1991**, *12*, 118–129. [\[CrossRef\]](#)
- Bergeretti, J.-F.; Carré, B.A. Information-flow and data-flow analysis of while-programs. *ACM Trans. Program. Lang. Syst.* **1985**, *7*, 37–61. [\[CrossRef\]](#)
- Knijnenburg, P.M.W.; Bik, A.J.C. On reducing overhead in loops. In Proceedings of the Fifth International Workshop on Compilers for Parallel Computers, Malaga, Spain, 26–30 June 1995.
- Tian, X.; Xu, R.; Yan, Y.; Chandrasekaran, S.; Eachempati, D.; Chapman, B. Compiler transformation of nested loops for general purpose GPUs. *Concurr. Comput. Pract. Exp.* **2016**, *28*, 537–556. [\[CrossRef\]](#)
- Sarkar, V. Optimized unrolling of nested loops. In Proceedings of the 14th International Conference on Supercomputing, Santa Fe, NM, USA, 8–11 May 2000.
- Koseki, A.; Komatsu, H.; Fukazawa, Y. A method for estimating optimal unrolling times for nested loops. In *Proceedings of the 1997 International Symposium on Parallel Architectures, Algorithms and Networks (I-SPAN'97)*; IEEE: Taipei, Taiwan, 1997.
- DeWitt, D.J.; Naughton, J.F.; Burger, J. Nested loops revisited. In *Proceedings of the Second International Conference on Parallel and Distributed Information Systems*; IEEE: San Diego, CA, USA, 1993.
- Collard, J.-F. Automatic parallelization of while-loops using speculative execution. *Int. J. Parallel Program.* **1995**, *23*, 191–219. [\[CrossRef\]](#)
- Baumann, P.; Misev, D.; Merticariu, V.; Huu, B.P. Array databases: Concepts, standards, implementations. *J. Big Data* **2021**, *8*, 28. [\[CrossRef\]](#)
- Scholz, S.B. Single Assignment C: Efficient Support for High-Level Array Operations in a Functional Setting. *J. Funct. Program.* **2003**, *13*, 1005–1059. [\[CrossRef\]](#)
- Gagniuć, P.A. Arrays. In *Coding Examples from Simple to Complex: Applications in JavaScript*; Springer: Cham, Switzerland, 2024; pp. 29–58.
- Pae, Y.; Hoeflinger, J.; Padua, D. Simplification of array access patterns for compiler optimizations. In Proceedings of the ACM SIGPLAN 1998 Conference on Programming Language Design and Implementation, Montreal, QC, Canada, 17–19 June 1998.
- Paek, Y.; Hoeflinger, J.; Padua, D. Efficient and precise array access analysis. *ACM Trans. Program. Lang. Syst. (TOPLAS)* **2002**, *24*, 65–109. [\[CrossRef\]](#)
- Balasundaram, V. A mechanism for keeping useful internal information in parallel programming tools: The data access descriptor. *J. Parallel Distrib. Comput.* **1990**, *9*, 154–170. [\[CrossRef\]](#)
- Gruber, B.M.; Amadio, G.; Blomer, J.; Matthes, A.; Widera, R.; Bussmann, M. LLAMA: The low-level abstraction for memory access. *Softw. Pract. Exp.* **2023**, *53*, 115–141. [\[CrossRef\]](#)
- Cichocki, A.; Lee, N.; Oseledets, I.; Phan, A.-H.; Zhao, Q.; Mandic, D.P. Tensor networks for dimensionality reduction and large-scale optimization: Part 1 low-rank tensor decompositions. *Found. Trends Mach. Learn.* **2016**, *9*, 249–429. [\[CrossRef\]](#)
- Lanzkron, P.J.; Rose, D.J.; Szyld, D.B. Convergence of nested classical iterative methods for linear systems. *Numer. Math.* **1990**, *58*, 685–702. [\[CrossRef\]](#)

24. Omran, H. Toward Memory-Efficient Analog Design Using Precomputed Lookup Tables. *Electronics* **2024**, *13*, 3776. [\[CrossRef\]](#)
25. Llanos, D.R.; Orden, D.; Palop, B. Just-in-time scheduling for loop-based speculative parallelization. In *16th Euromicro Conference on Parallel, Distributed and Network-Based Processing (PDP 2008)*; IEEE: Toulouse, France, 2008.
26. Estebanez, A.; Llanos, D.R.; Orden, D.; Palop, B. On the choice of the best chunk size for the speculative execution of loops. *PLoS ONE* **2022**, *17*, e0267602. [\[CrossRef\]](#) [\[PubMed\]](#)
27. Owens, J.D.; Houston, M.; Luebke, D.; Green, S.; Stone, J.E.; Phillips, J.C. GPU computing. *Proc. IEEE* **2008**, *96*, 879–899. [\[CrossRef\]](#)
28. Nickolls, J.; Dally, W.J. The GPU computing era. *IEEE Micro* **2010**, *30*, 56–69. [\[CrossRef\]](#)
29. Brodtkorb, A.R.; Hagen, T.R.; Sætra, M.L. Graphics processing unit (GPU) programming strategies and trends in GPU computing. *J. Parallel Distrib. Comput.* **2013**, *73*, 4–13. [\[CrossRef\]](#)
30. Charlier, B.; Feydy, J.; Glaunes, J.A.; Collin, F.D.; Durif, G. Kernel operations on the GPU, with autodiff, without memory overflows. *J. Mach. Learn. Res.* **2021**, *22*, 1–6.
31. Shiue, L.J.; Jones, I.; Peters, J. A realtime GPU subdivision kernel. *ACM Trans. Graph. (TOG)* **2005**, *24*, 1010–1015. [\[CrossRef\]](#)
32. Monmasson, E.; Cirstea, M.N. FPGA design methodology for industrial control systems—A review. *IEEE Trans. Ind. Electron.* **2007**, *54*, 1824–1842. [\[CrossRef\]](#)
33. Boutros, A.; Betz, V. FPGA architecture: Principles and progression. *IEEE Circuits Syst. Mag.* **2021**, *21*, 4–29. [\[CrossRef\]](#)
34. Mittal, S.; Vetter, J.S. A survey of CPU-GPU heterogeneous computing techniques. *ACM Comput. Surv. (CSUR)* **2015**, *47*, 1–35. [\[CrossRef\]](#)
35. Memon, Z.A.; Samad, F.; Awan, Z.R.; Aziz, A.; Siddiqi, S.S. Cpu-gpu processing. *Int. J. Comput. Sci. Netw. Secur.* **2017**, *17*, 188–193.
36. Yıldız, A.; Ugurdag, H.F.; Aktemur, B.; İskender, D.; Gören, S. CPU design simplified. In *2018 3rd International Conference on Computer Science and Engineering (UBMK)*; IEEE: Sarajevo, Bosnia and Herzegovina, 2018.
37. Ryoo, S.; Rodrigues, C.I.; Baghsorkhi, S.S.; Stone, S.S.; Kirk, D.B.; W. Hwu., W. Optimization principles and application performance evaluation of a multithreaded GPU using CUDA. In *Proceedings of the 13th ACM SIGPLAN Symposium on Principles and practice of parallel programming (PPoPP '08)*; Association for Computing Machinery: New York, NY, USA, 2008; pp. 73–82.
38. Fatahalian, K.; Houston, M. A closer look at GPUs. *Commun. ACM* **2008**, *51*, 50–57. [\[CrossRef\]](#)
39. Li, A.; Liu, W.; Wang, L.; Barker, K.; Song, S.L. Warp-consolidation: A novel execution model for GPUs. In *Proceedings of the 2018 International Conference on Supercomputing*, Beijing, China, 12–15 June 2018.
40. Brunie, N.; Collange, C.; Damos, G. Simultaneous branch and warp interweaving for sustained GPU performance. *IEEE Xplore* **2012**, *40*, 49–60.
41. Gagniuc, P.A.; Ionescu-Tirgoviste, C.; Gagniuc, E.; Militaru, M.; Nwabudike, L.C.; Pavaloio, B.I.; Vasilăţeanu, A.; Goga, N.; Drăgoi, G.; Popescu, I.; et al. Spectral forecast: A general purpose prediction model as an alternative to classical neural networks. *Chaos* **2020**, *30*, 033119. [\[CrossRef\]](#)
42. Ionescu-Tirgoviste, C.; Gagniuc, P.A.; Gagniuc, E. The electrical activity map of the human skin indicates strong differences between normal and diabetic individuals: A gateway to onset prevention. *Biosens. Bioelectron.* **2018**, *120*, 188–194. [\[CrossRef\]](#)
43. Anghelache, F.; Marian, C.V.; Mitrea, D.A.; Goga, N.; Vasilăţeanu, A.; Rădulescu, V.; Muşat, D.; Scurtu, D. iRoute-An Adaptive Route Planning Solution for Commercial Vehicle Fleets. *Appl. Sci.* **2023**, *13*, 11517. [\[CrossRef\]](#)
44. Anghelache, F.; Goga, N.; Marian, C.V.; Mitrea, D.A.; Păvăloiu, I.-B. Usability and Perceived Efficiency of an Adaptive Route Optimization Solution for Commercial Vehicles. *IEEE Access* **2025**, *13*, 61091–61108. [\[CrossRef\]](#)
45. Hobincu, R.; Codreanu, V.; Petrica, L. GNU Compiler Collection Backend Port for the Integral Parallel Architecture. *UPB Sci. Bull. Ser. C* **2012**, *74*, 79–92.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.